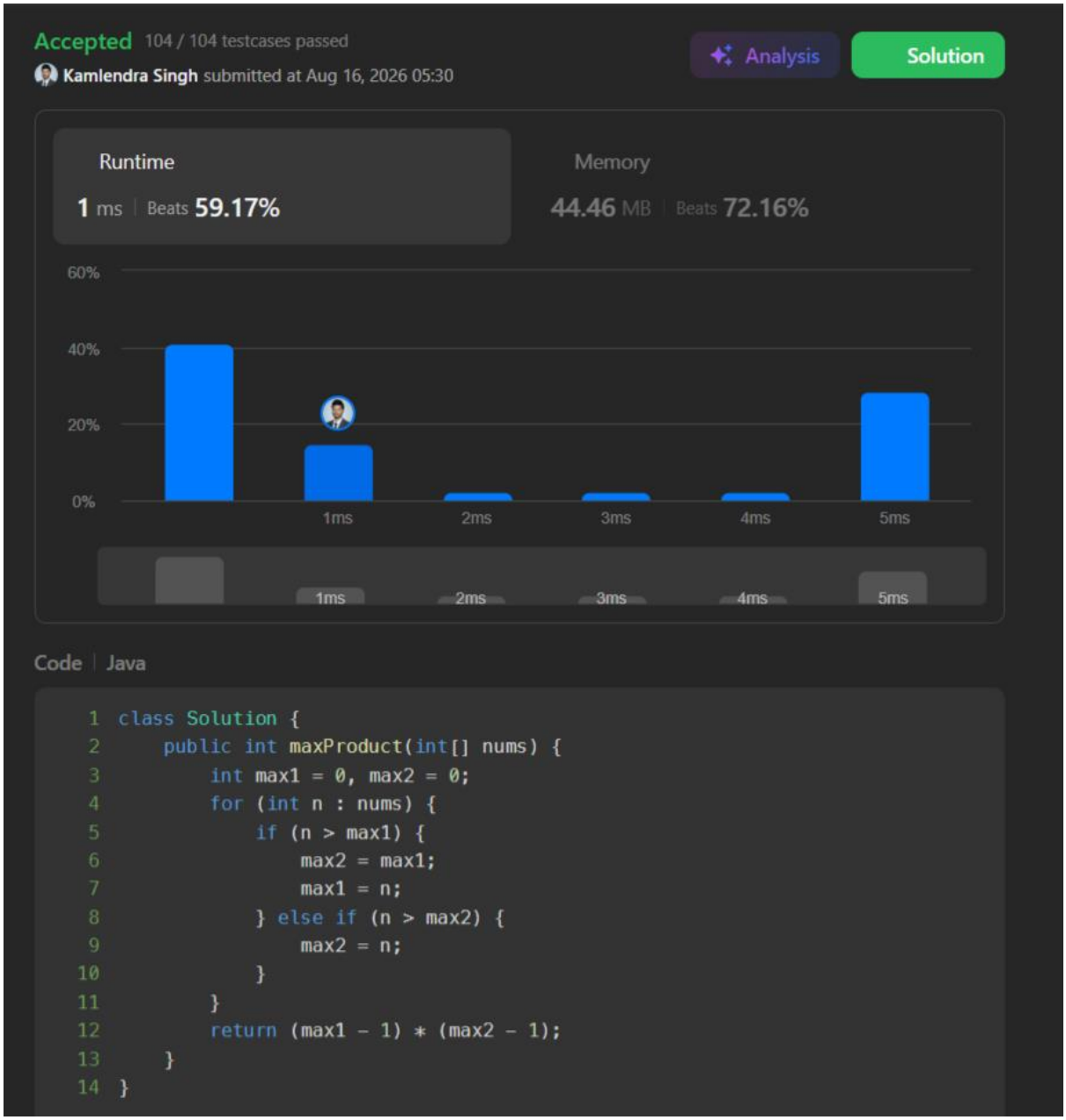
Problem Name:

1464. Maximum Product of Two Elements in an Array

Accepted code:

```
class Solution {
    public int maxProduct(int[] nums) {
        int max1 = 0, max2 = 0;
        for (int n : nums) {
            if (n > max1) {
                max2 = max1;
                max1 = n;
            } else if (n > max2) {
                max2 = n;
            }
        }
        return (max1 - 1) * (max2 - 1);
    }
}
```



Time Complexity: O(n)

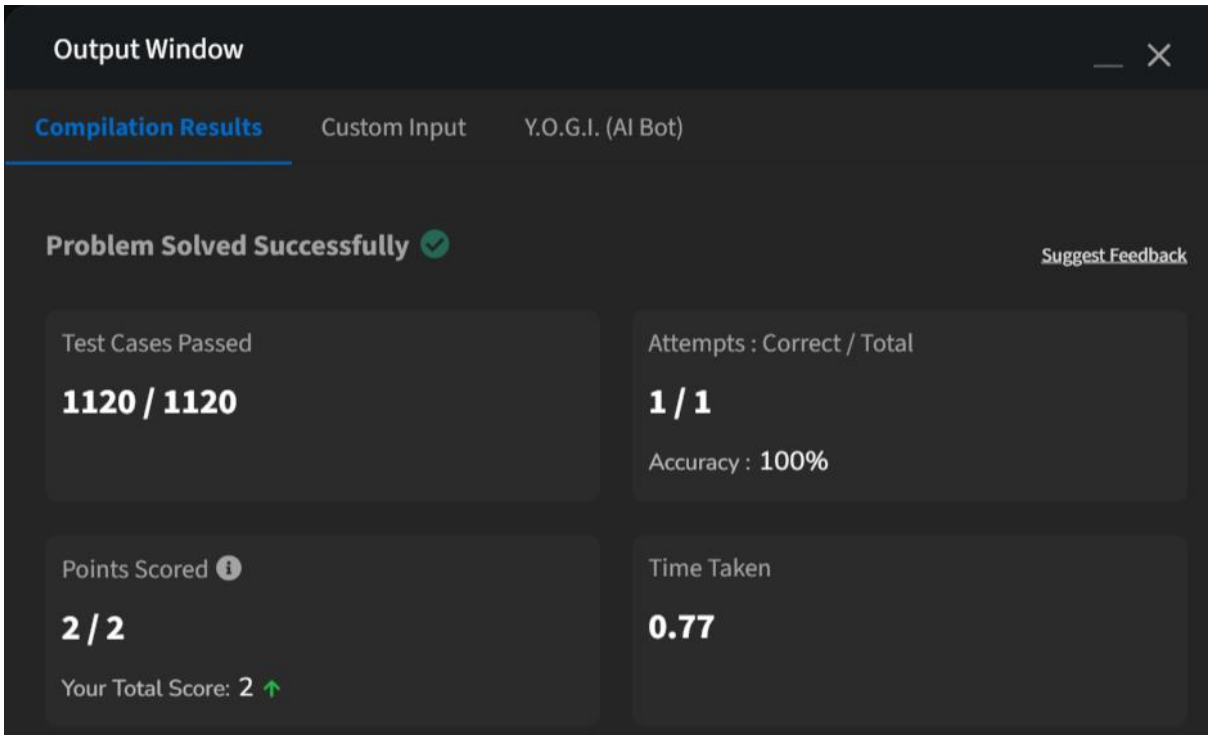
Problem Name:

Second Largest

Accepted code :

```
class Solution {
    public int getSecondLargest(int[] arr) {
        int first = -1, second = -1;
        for (int x : arr) {
```

```
            if (x > first) {
                second = first;
                first = x;
            } else if (x > second && x != first) {
                second = x;
            }
        }
        return second;
    }
}
```



Time Complexity: O(n)

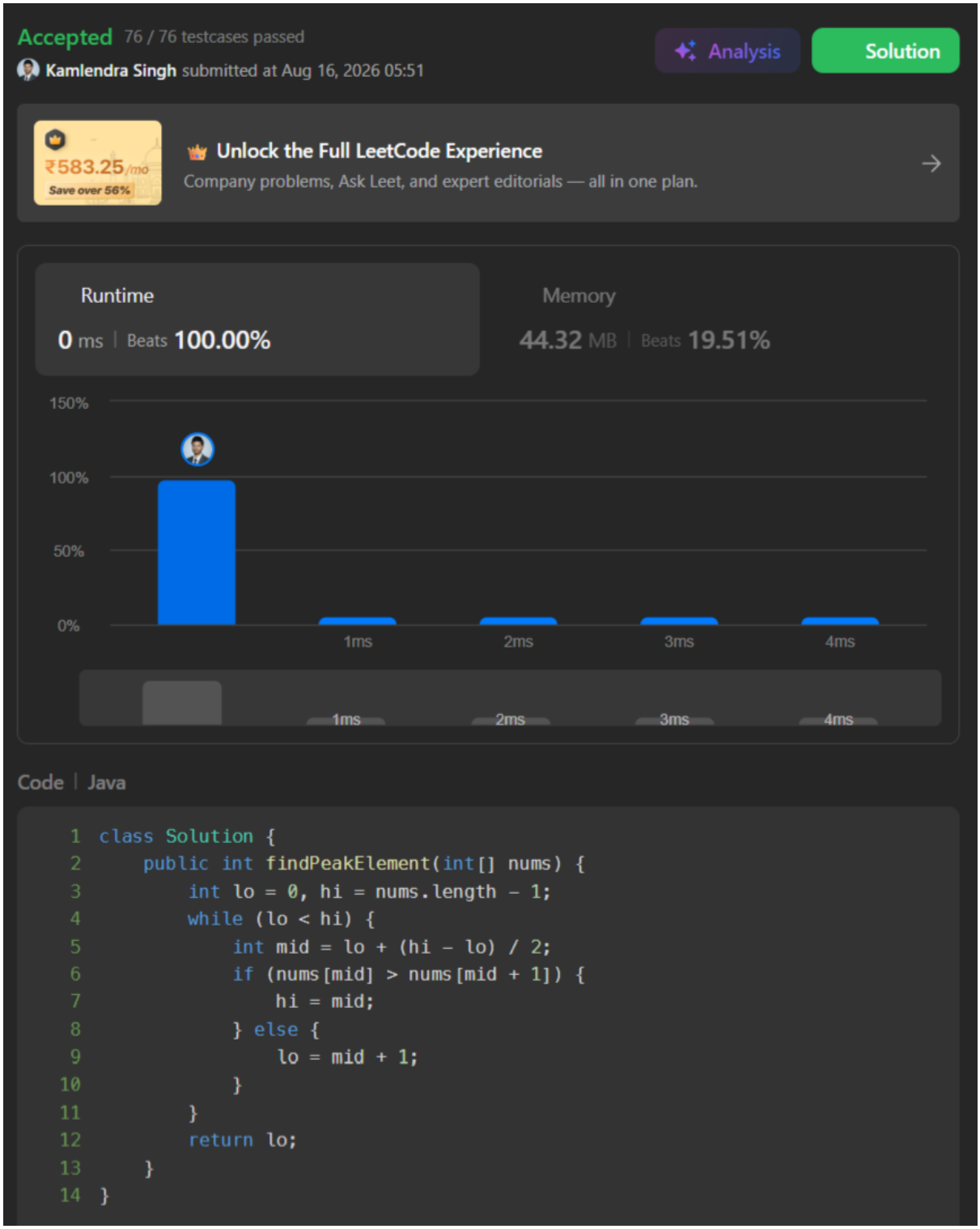
Problem Name:

162. Find Peak Element

Accepted code:

```
class Solution {
    public int findPeakElement(int[] nums) {
```

```
        int lo = 0, hi = nums.length - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] > nums[mid + 1]) {
                hi = mid;
            } else {
                lo = mid + 1;
            }
        }
        return lo;
    }
}
```



Time Complexity: O(log n)

Problem Name:

33. Search in Rotated Sorted Array

Accepted code:

class Solution {

```
public int search(int[] nums, int target) {
    int low = 0;
    int high = nums.length - 1;

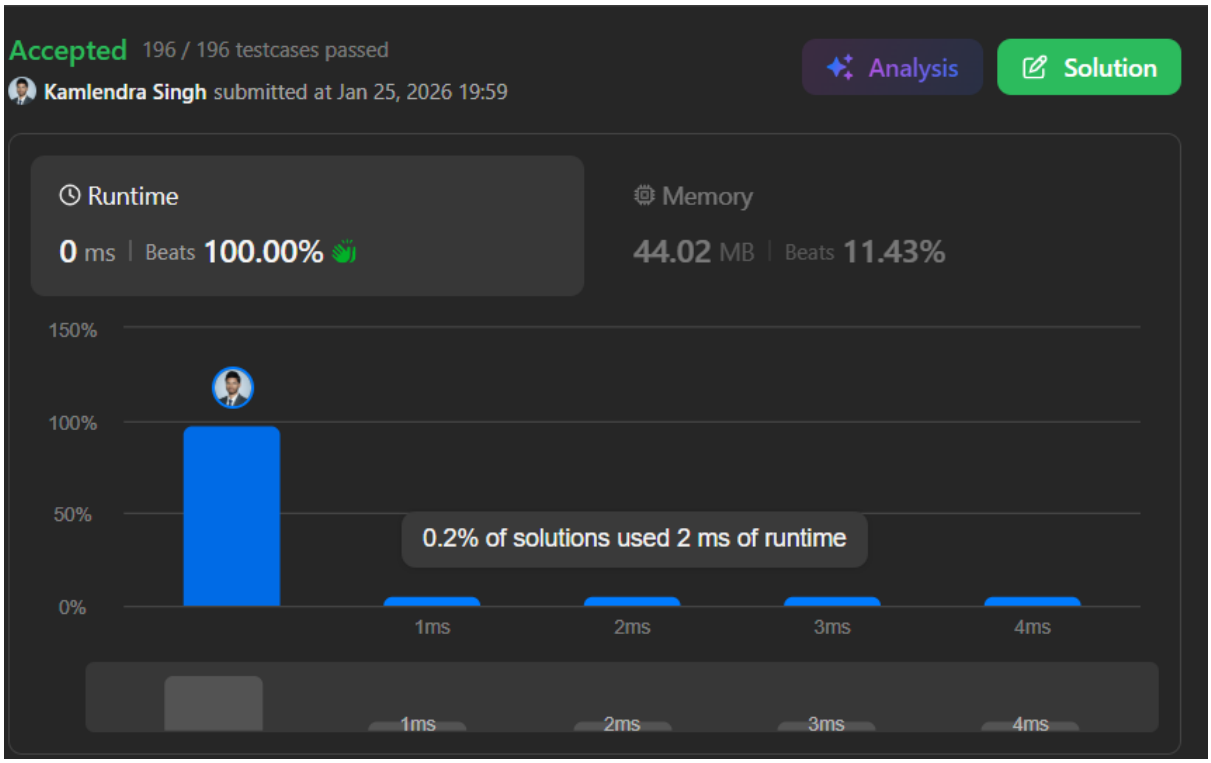
    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (nums[mid] == target) {
            return mid;
        }

        if (nums[low] <= nums[mid]) {
            if (target >= nums[low] && target < nums[mid]) {
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        } else {
            if (target > nums[mid] && target <= nums[high]) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
    }

    return -1;
}
```

}



Time Complexity: O(log n)

Problem Name:

3477. Fruits Into Baskets II

Accepted code:

```
class Solution {
    public int numOfUnplacedFruits(int[] fruits, int[] baskets) {
        int n = fruits.length;
        boolean[] used = new boolean[n];
        int unplaced = 0;
        for (int i = 0; i < n; i++) {
            boolean placed = false;
            for (int j = 0; j < n; j++) {
                if (!used[j] && baskets[j] >= fruits[i]) {
                    used[j] = true;
                    placed = true;
```

```
                break;
            }
        }
        if (!placed) {
            unplaced++;
        }
    }
    return unplaced;
  }
}
```



Time Complexity: $O(n^2)$

Problem Name:

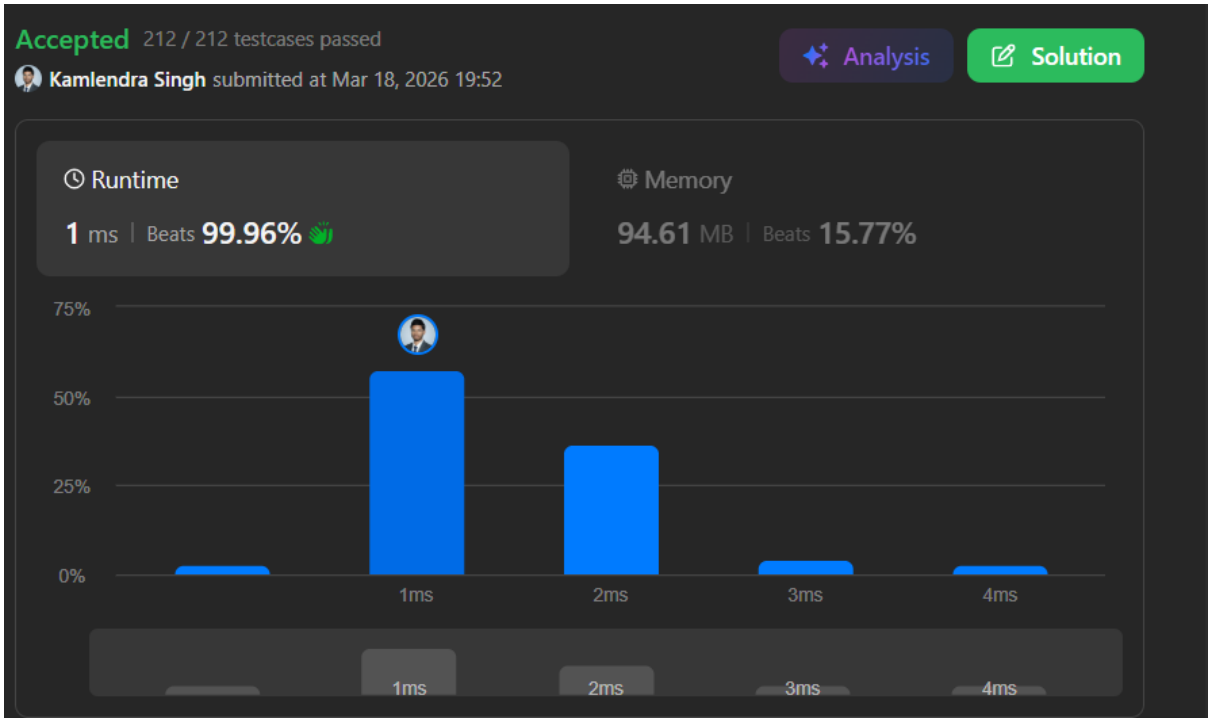
Best Time to Buy and Sell Stock

Accepted code:

```
class Solution {
    public int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE;
        int maxProfit = 0;

        for (int price : prices) {
            if (price < minPrice) {
                minPrice = price;
            } else if (price - minPrice > maxProfit) {
                maxProfit = price - minPrice;
            }
        }

        return maxProfit;
    }
}
```



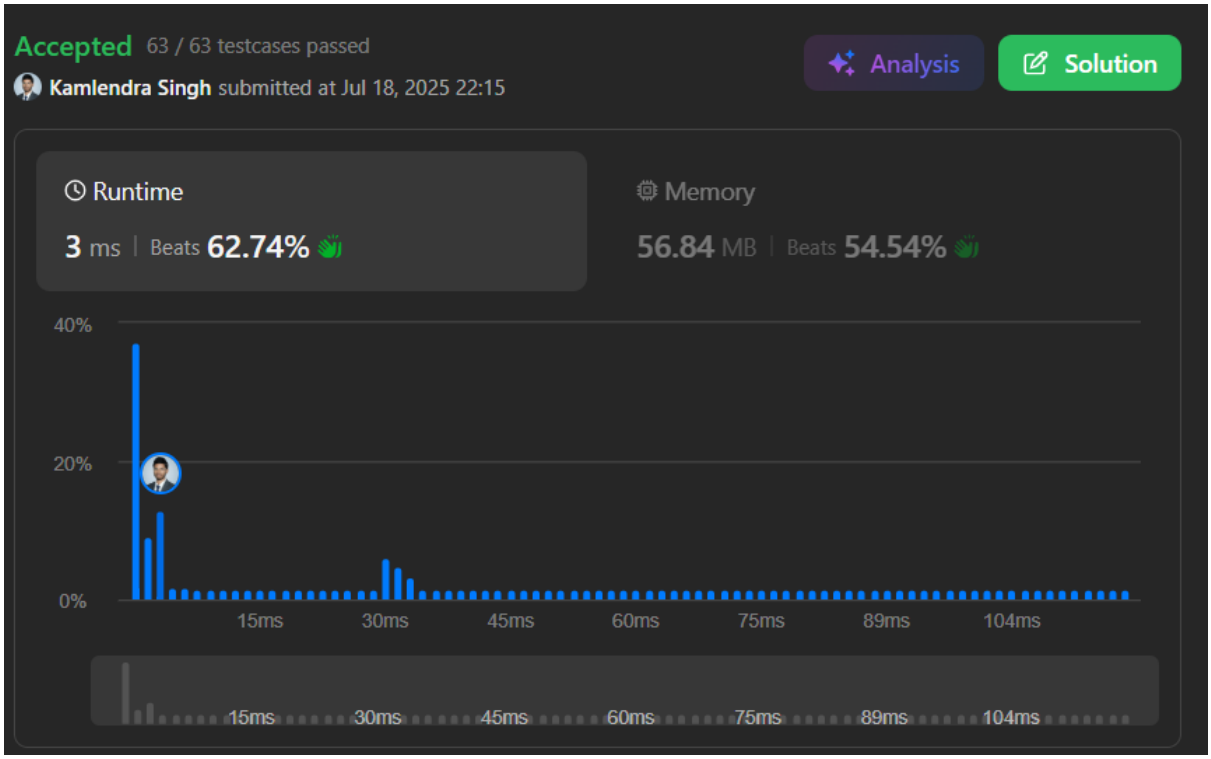
Time Complexity: O(n)

Problem Name:

Two Sum

Accepted code:

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] + nums[j] == target) {
                    return new int[]{i, j};
                }
            }
        }
        return new int[]{-1, -1};
    }
}
```



Time Complexity: $O(n^2)$

Problem Name:

1365. How Many Numbers Are Smaller Than the Current Number

Accepted code:

```
class Solution {
    public int[] smallerNumbersThanCurrent(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        for (int i = 0; i < n; i++) {
            int count = 0;
            for (int j = 0; j < n; j++) {
                if (j != i && nums[j] < nums[i]) {
                    count++;
                }
            }
            result[i] = count;
        }
        return result;
    }
}
```



Time Complexity: $O(n^2)$

Problem Name:

628. Maximum Product of Three Numbers

Accepted code:

```
class Solution {
    public int maximumProduct(int[] nums) {
        int max1 = Integer.MIN_VALUE, max2 = Integer.MIN_VALUE, max3 = Integer.MIN_VALUE;
        int min1 = Integer.MAX_VALUE, min2 = Integer.MAX_VALUE;

        for (int n : nums) {
            if (n > max1) {
                max3 = max2;
                max2 = max1;
                max1 = n;
```

```
            } else if (n > max2) {
                max3 = max2;
                max2 = n;
            } else if (n > max3) {
                max3 = n;
            }

            if (n < min1) {
                min2 = min1;
                min1 = n;
            } else if (n < min2) {
                min2 = n;
            }
        }

        return Math.max(max1 * max2 * max3, max1 * min1 * min2);
    }
}
```



Time Complexity: O(n)

Problem Name:

26. Remove Duplicates from Sorted Array

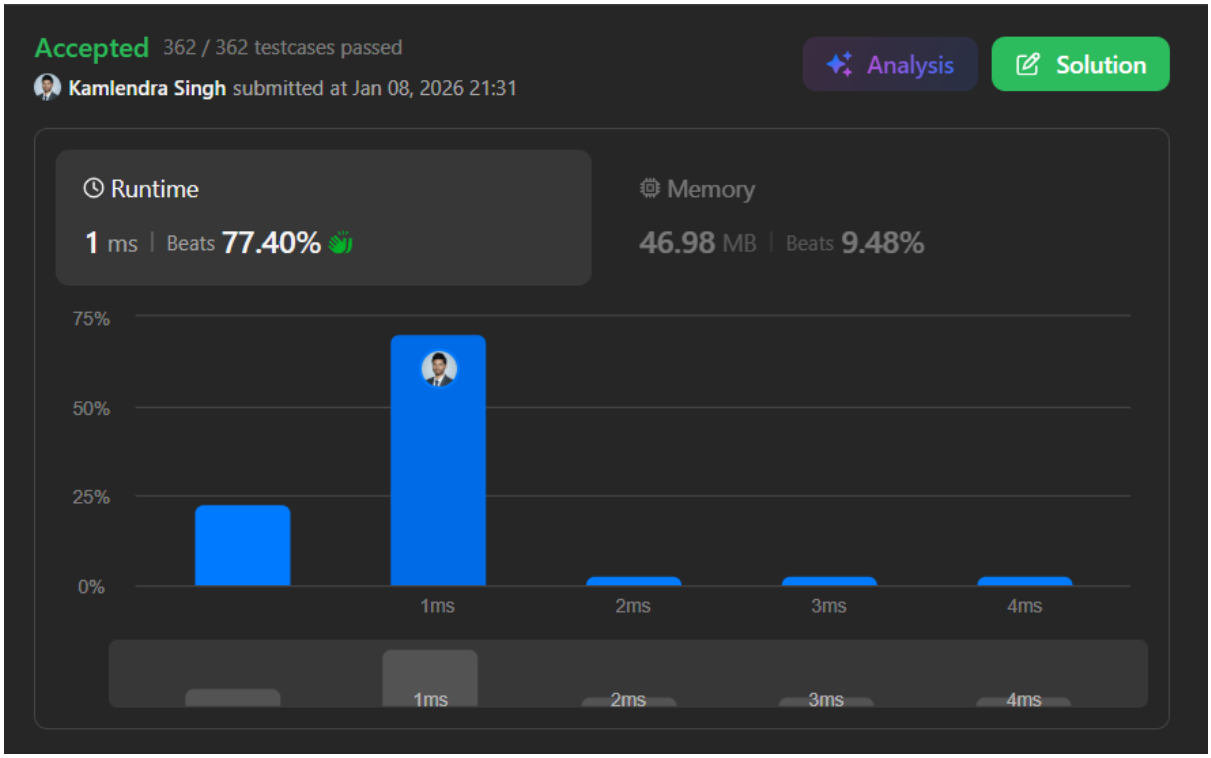
Accepted code:

```
class Solution {
    public int removeDuplicates(int[] nums) {
        if (nums.length == 0) return 0;

        int i = 0;
        for (int j = 1; j < nums.length; j++) {
            if (nums[j] != nums[i]) {
                i++;
                nums[i] = nums[j];
            }
```

```
        }

        return i + 1;
    }
}
```



Time Complexity: O(n)